

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERÍA EN CIENCIAS Y SISTEMAS
Laboratorio Organización de lenguajes y compiladores 1
SECCIÓN C
PRIMER SEMESTRE 2026



Manual tecnico GoLite

Nombre :Adilzon Alfredo Velásquez Hernández
Carné :201908076

Guatemala, Sabado 28 de Junio del 2026

Introducción

Este manual técnico describe la arquitectura, diseño e implementación del intérprete del lenguaje GoLite, desarrollado como proyecto para el curso de Organización de Lenguajes y Compiladores 1 en la Universidad San Carlos de Guatemala.

El proyecto consiste en un intérprete completo que incluye análisis léxico, análisis sintáctico, construcción del AST (Árbol de Sintaxis Abstracta) y un intérprete que recorre el AST para ejecutar el código fuente escrito en GoLite.

2. Requisitos del Sistema

Componente	Versión	Descripción
Java JDK	21 o superior	Lenguaje base del proyecto
Maven	3.8+	Gestor de dependencias y construcción
JFLEX	1.9.1	Generación del analizador léxico (vía plugin Maven)
CUP	11b-20160615-3	Generación del analizador sintáctico (vía plugin Maven)
RSyntaxTextArea	3.3.4	Editor con resaltado de sintaxis en la GUI

3. Estructura del Proyecto

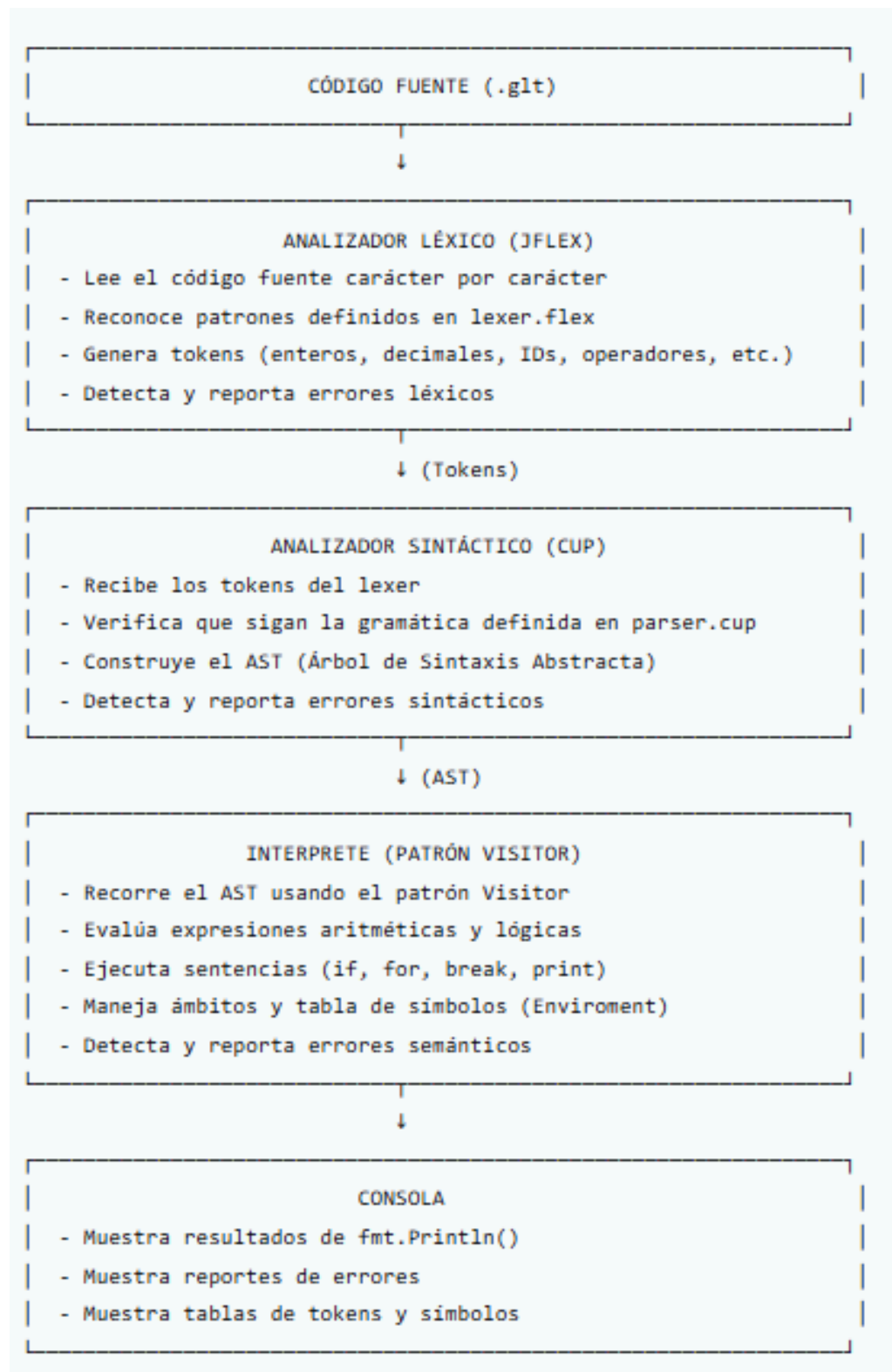
clase06/

```
|— pom.xml          # Configuración Maven
|— README.md        # Descripción del proyecto
|— Manual_Tecnico.pdf      # Este documento
|— Manual_Usuario.pdf     # Manual de usuario
|
|_ src/
    |_ main/
        |_ java/olc1/golite/
            | |— Main.java      # Punto de entrada
            | |
            | |— ast/          # Nodos del AST
            | |   |_ ASTNode.java  # Interfaz base
            | |   |_ exp/        # Nodos de expresiones
            | |       |_ Add.java   # Suma
            | |       |_ Sub.java   # Resta
            | |       |_ Mul.java   # Multiplicación
            | |       |_ Div.java   # División
            | |       |_ Lt.java    # Menor que
            | |       |_ Negate.java # Negación unaria
            | |       |_ Integers.java # Literal entero
            | |       |_ Decimal.java # Literal decimal
            | |       |_ BoolLiteral.java # Literal booleano
            | |       |_ StringLiteral.java # Literal cadena
```

- | | | └─ VarRef.java # Referencia a variable
- | | └─ stm/ # Nodos de sentencias
- | | └─ Assign.java # Asignación
- | | └─ Allocate.java # Reasignación
- | | └─ Imprimir.java # Print
- | | └─ IfNode.java # If-else
- | | └─ ElifNode.java # Else if
- | | └─ ElifNodes.java # Lista de elif
- | | └─ WhileFor.java # Bucle for
- | | └─ Breakstm.java # Break
- | | └─ Statments.java # Bloque de sentencias
- | |
- | └─ reports/ # Manejo de errores
- | | └─ GoliteError.java # Estructura de error
- | |
- | └─ views/ # Interfaz gráfica
- | | └─ GoliteFrame.java # Ventana principal
- | | └─ EditorPanel.java # Panel del editor
- | | └─ GoliteMenuBar.java # Barra de menú
- | | └─ MainPanel.java # Panel principal (split)
- | |
- | └─ visitor/ # Patrón Visitor
- | └─ Visitor.java # Interfaz Visitor
- | └─ interpreter/ # Visitor intérprete
- | └─ InterpreterVisitor.java

```
|      |— Enviroment.java
|      |— transfer/
|      |  └─ BreakException.java
|      └─ value/
|          |— ValueWrapper.java
|          |— IntValue.java
|          |— DecimalValue.java
|          |— BoolValue.java
|          |— StringValue.java
|          └─ VoidValue.java
|
|— jflex/
|  └─ lexer.flex      # Especificación del lexer
|
└─ cup/
    └─ parser.cup      # Especificación del parser
```

4. Diagrama de Arquitectura



Generación de Analizadores

5.1 JFLEX (Analizador Léxico)

Archivo: `src/main/jflex/lexer.flex`

Para esta parte, utilicé la librería JFLEX porque es la herramienta estándar para generar analizadores léxicos en Java. Decidí usarla en lugar de implementar un lexer manualmente porque:

- Reduce drásticamente el código manual
- Es más fácil de mantener y modificar
- Se integra bien con Maven mediante el plugin `jflex-maven-plugin`

Patrones definidos en el lexer:

Categoría	Patrón	Token generado	Ejemplo
Enteros	<code>[0-9]+</code>	<code>integer</code>	123, 45
Decimales	<code>[0-9]+\.[0-9]+</code>	<code>decimal</code>	3.14, 2.5
Identificadores	<code>[a-zA-Z_][a-zA-Z0-9_]*</code>	<code>id</code>	x, contador, _temp
Cadenas	<code>"([^\\"] \\.)*"</code>	<code>string</code>	"Hola mundo"
Operador suma	<code>"+"</code>	<code>plus</code>	+
Operador resta	<code>"-"</code>	<code>minus</code>	-
Operador multiplicación	<code>"*"</code>	<code>times</code>	*
Operador división	<code>"/"</code>	<code>slash</code>	/

Operador menor que	"<"	lt	<
Asignación	":="	assign	:=
Reasignación	"="	allocate	=
Punto y coma	";"	scol	;
Paréntesis izquierdo	"("	lparen	(
Paréntesis derecho	")"	rparen	)
Llave izquierda	"{"	lbrace	{
Llave derecha	"}"	rbrace	}
Palabra reservada	"print"	imprimir	print
		print	
Palabra reservada	"true"	kwTrue	true
		true	
Palabra reservada	"false"	kwFalse	false
		false	
Palabra reservada if	"if"	kwIf	if
Palabra reservada	"else"	kwElse	else
		else	
Palabra reservada	"for"	kwFor	for
		for	

Palabra reservada

"break"

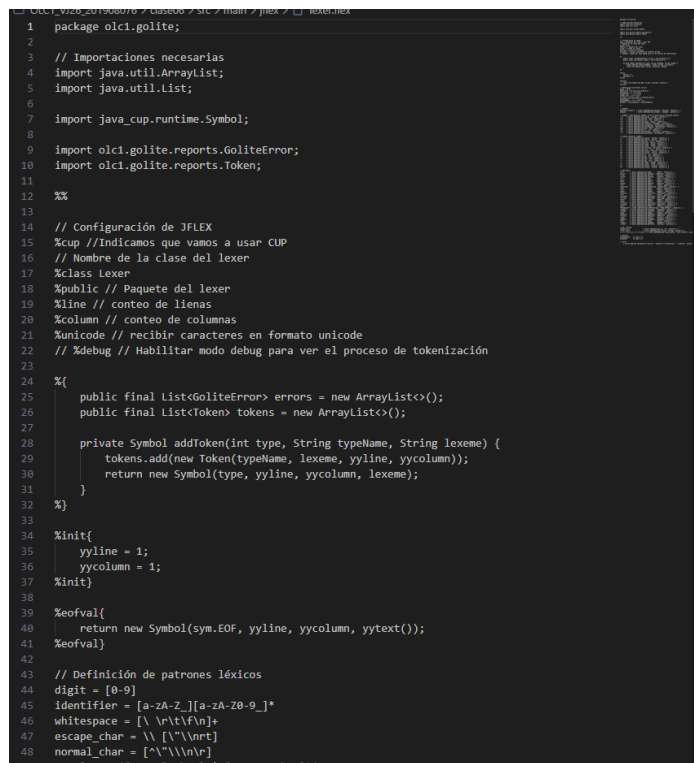
kwBreak

break

break

Manejo de errores léxicos:

Cuando el lexer encuentra un carácter no reconocido, crea un objeto `GoliteError` con tipo "Léxico", una descripción y la posición (línea, columna). Esto permite mostrar todos los errores al final del análisis.



```
1 package olci.golite;
2
3 // Importaciones necesarias
4 import java.util.ArrayList;
5 import java.util.List;
6
7 import java_cup.runtime.Symbol;
8
9 import olci.golite.reports.GoliteError;
10 import olci.golite.reports.Token;
11
12 %%
13
14 // Configuración de JFLEX
15 %cup //Indicamos que vamos a usar CUP
16 // Nombre de la clase del lexer
17 %class Lexer
18 %public // Paquete del lexer
19 %line // conteo de líneas
20 %column // conteo de columnas
21 %unicode // recibir caracteres en formato unicode
22 // %debug // Habilitar modo debug para ver el proceso de tokenización
23
24 %{
25     public final List<GoliteError> errors = new ArrayList<>();
26     public final List<Token> tokens = new ArrayList<>();
27
28     private Symbol addToken(int type, String typeName, String lexeme) {
29         tokens.add(new Token(typeName, lexeme, yylne, yycolumn));
30         return new Symbol(type, yylne, yycolumn, lexeme);
31     }
32 }%
33
34 %init{
35     yylne = 1;
36     yycolumn = 1;
37 %init}
38
39 %eofval{
40     return new Symbol(sym.EOF, yylne, yycolumn, yytext());
41 %eofval}
42
43 // Definición de patrones léxicos
44 digit = [0-9]
45 identifier = [a-zA-Z][a-zA-Z0-9_]*
46 whitespace = [\ \r\t\v\n]+
47 escape_char = \\ [^\nrt]
48 normal_char = [^"\\\n\r]
49 %token <{normal_char}|{escape_char}>
```

5.2 CUP (Analizador Sintáctico)

Archivo: `src/main/cup/parser.cup`

Para el análisis sintáctico, utilicé CUP (Constructor of Useful Parsers) porque es la herramienta complementaria a JFLEX y genera parsers LALR. Decidí usarla porque:

- Genera el parser automáticamente a partir de la gramática
- Se integra con JFLEX (ambos usan la misma clase `sym` para los tokens)
- Produce un AST con acciones semánticas embebidas

Gramática del lenguaje (BNF):

programa ::= instrucciones

instrucciones ::= instruccion instrucciones

| instruccion

instruccion ::= imprimir lparen expresion rparen scol

| id assign expresion scol

| id allocate expresion scol

| kwIf expresion lbrace instrucciones rbrace

| kwIf expresion lbrace instrucciones rbrace ELIF

| kwFor expresion lbrace instrucciones rbrace

| kwBreak scol

| kwBreak

| error

expresion ::= expresion plus expresion

| expresion minus expresion

| expresion times expresion

| expresion slash expresion

| expresion lt expresion

| minus expresion

| lparen expresion rparen

| kwTrue

| kwFalse

| id

| integer

| decimal

| string

```

1 package olci.golite;
2
3 // importaciones necesarias
4 import java.util.ArrayList;
5 import java.util.List;
6
7 import java_cup.runtime.Symbol;
8
9 import olci.golite.ast.ASTNode;
10 import olci.golite.ast.exp.*;
11 import olci.golite.ast.stm.*;
12 import olci.golite.reports.GoliteError;
13
14 parser code
15 {
16     public final List<GoliteError> errors = new ArrayList<>();
17
18     public void syntax_error(Symbol s) {
19         errors.add(new GoliteError("Sintactico", "Simbolo inesperado: " + s.value, s.left
20     }
21
22     public void unrecovered_syntax_error(Symbol s) {
23         errors.add(new GoliteError("Sintactico", "Simbolo inesperado no recuperable: " +
24     }
25 }
26
27 action code
28 {
29     String consola = "";
30 }
31
32 terminal String string, integer, decimal, rune_literal;
33 terminal lparen, rparen, lbrace, rbrace, lbrack, rbrack;
34 terminal plus, minus, times, slash, UMENOS;
35 terminal scol, assign, allocate;
36 terminal String id;
37 terminal imprimir, kwTrue, kwFalse, kwIf, kwElse;
38 terminal kwFor, kwBreak;
39 terminal lt, gte, lte, gt, equal, notequal, mod, and, or, not;
40 terminal plusAssign, minusAssign, dot;
41 terminal kwVar, kwContinue, kwFct, kwFunc, kwMain, kwPrintln;
42 terminal kwInt, kwFloat64, kwString, kwBool, kwRune, kwNil;
43 terminal kwStrconv, kwReflect, kwAtoi, kwParseFloat, kwTypeOf;
44 terminal plusPlus, minusMinus;
45 terminal comma;
46 terminal kwSwitch, kwCase, kwDefault, kwReturn, kwType, kwAppend, kwLen, kwRange;
47 terminal FUNC, TYPE, SWITCH, CASE, DEFAULT, RETURN, APPEND, LEN, RANGE;
48 terminal kwStruct;
49 terminal colon;

```

Acciones semánticas:

Cada regla de producción en CUP incluye una acción semántica que construye el nodo correspondiente del AST. Por ejemplo:

instruccion ::= imprimir lparen expr rparen scol

```
{: RESULT = new Imprimir(expr); :}
```

Esta acción crea un objeto `Imprimir` que contiene la expresión a mostrar.

6. Árbol de Sintaxis Abstracta (AST)

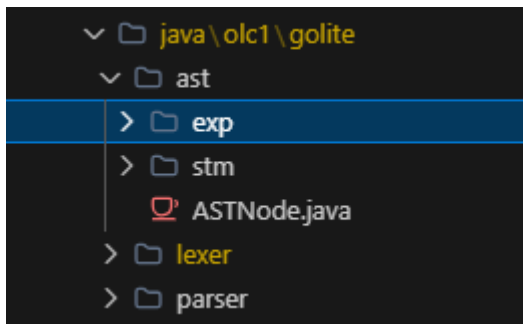
El AST es la representación jerárquica del código fuente después del análisis sintáctico. Decidí implementar el AST usando el patrón Composite combinado con el patrón Visitor por las siguientes razones:

1. Separación de preocupaciones: Los nodos solo almacenan datos, la lógica de interpretación está en el Visitor
2. Extensibilidad: Es fácil agregar nuevas operaciones (como generar código, validar tipos) sin modificar los nodos
3. Mantenibilidad: El código es más limpio y organizado

6.1 Interfaz Base ASTNode

```
public interface ASTNode {  
    <T> T accept(Visitor<T> visitor);  
}
```

Todos los nodos del AST implementan esta interfaz. El método `accept` recibe un `Visitor` y delega la visita al método correspondiente.



6.2 Nodos de Expresión

Clase	Representación	Ejemplo GoLite
Add	Suma	a + b
Sub	Resta	a - b
Mul	Multiplicación	a * b
Div	División	a / b

<code>Lt</code>	Menor que	<code>a < b</code>
<code>Negate</code>	Negación unaria	<code>-a</code>
<code>Integers</code>	Literal entero	<code>42</code>
<code>Decimal</code>	Literal decimal	<code>3.1416</code>
<code>BoolLiteral</code>	Literal booleano	<code>true / false</code>
<code>StringLiteral</code>	Literal cadena	<code>"texto"</code>
<code>VarRef</code>	Referencia a variable	<code>x</code>

6.3 Nodos de Sentencia

Clase	Representación	Ejemplo GoLite
<code>Assign</code>	Declaración con asignación	<code>x := 10</code>
<code>Allocate</code>	Reasignación	<code>x = 20</code>

Imprimir

Impresión

print(x)

IfNode

Condicional

if cond { ... }

ElifNode

Else if

else if cond { ... }

ElifNodes

Lista de elifs

else if ... else if ...

WhileFor

Bucle for

for cond { ... }

Breakstm

Break

break

Statments

Bloque de sentencias

{ ... }

```
OLC1_VJ26_201908076 > clase06 > src > main > java > olc1 > golite > ast > ASTNode.java > {} olc1.golite.ast
1  package olc1.golite.ast;
2
3  import olc1.golite.visitor.Visitor;
4
5  // Interfaz base para todos los nodos de mi Árbol de Sintaxis Abstracta (AST).
6  // Define el método accept para implementar la doble distribución del patrón Visitor.
7  public interface ASTNode {
8      <T> T accept(Visitor<T> visitor);
9  }
10
```

7. Patrón Visitor

El patrón Visitor es fundamental en este proyecto. Decidí implementarlo porque:

- Permite separar los algoritmos (interpretación) de la estructura de datos (AST)
- Es fácil agregar nuevas operaciones sin modificar las clases de los nodos
- Es el patrón estándar para recorrer AST en compiladores e intérpretes

7.1 Interfaz Visitor

```
public interface Visitor<T> {  
  
    T visit(Integers.Context ctx);  
  
    T visit(Decimal.Context ctx);  
  
    T visit(Add.Context ctx);  
  
    T visit(Sub.Context ctx);  
  
    T visit(Mul.Context ctx);  
  
    T visit(Div.Context ctx);  
  
    T visit(Lt.Context ctx);  
  
    T visit(Negate.Context ctx);  
  
    T visit(VarRef.Context ctx);  
  
    T visit(Assign.Context ctx);  
  
    T visit(Allocate.Context ctx);  
  
    T visit(Imprimir.Context ctx);  
  
    T visit(IfNode.Context ctx);  
  
    T visit(WhileFor.Context ctx);  
  
    T visit(Breakstm.Context ctx);  
  
    T visit(Statments.Context ctx);  
  
    // ... más métodos según los nodos
```

}

```
1 package olci.golite.visitor;
2
3 import olci.golite.ast.exp.*;
4 import olci.golite.ast.stm.*;
5
6 // Interfaz base del patrón Visitor.
7 // Define la firma de los métodos 'visit' para cada nodo concreto de mi estructura AST.
8 public interface Visitor<T> {
9     T visit(Integers.Context ctx);
10    T visit(Decimal.Context ctx);
11    T visit(Paren.Context ctx);
12    T visit(Add.Context ctx);
13    T visit(Sub.Context ctx);
14    T visit(Mul.Context ctx);
15    T visit(Div.Context ctx);
16    T visit(Negate.Context ctx);
17    T visit(BoolLiteral.Context ctx);
18    T visit(StringLiteral.Context ctx);
19    T visit(VarRef.Context ctx);
20    T visit(Imprimr.Context ctx);
21    T visit(Assign.Context ctx);
22    T visit(IfNode.Context ctx);
23    T visit(Statements.Context ctx);
24    T visit(ElifNode.Context ctx);
25    T visit(ElifNodes.Context ctx);
26    T visit(Allocate.Context ctx);
27    T visit(Lt.Context ctx);
28    T visit(Breakstm.Context ctx);
29    T visit(WhileFor.Context ctx);
30    T visit(Gt.Context ctx);
31    T visit(Gte.Context ctx);
32    T visit(Lte.Context ctx);
33    T visit(Equal.Context ctx);
34    T visit(NotEqual.Context ctx);
35    T visit(Mod.Context ctx);
36    T visit(And.Context ctx);
37    T visit(Or.Context ctx);
38    T visit(Not.Context ctx);
39    T visit(Continuestm.Context ctx);
40    T visit(Atoi.Context ctx);
41    T visit(ParseFloat.Context ctx);
42    T visit(TypeOf.Context ctx);
43    T visit(BlockStm.Context ctx);
44    T visit(ForClassico.Context ctx);
45    T visit(NullLiteral.Context ctx);
46    T visit(NullLiteral.Context ctx);
47    T visit(SwitchNode.Context ctx);
48    T visit(CaseNode.Context ctx);
49 }
```

7.2 Contextos Internos

Cada nodo del AST define una clase interna `Context` que encapsula los datos del nodo. Decidí usar este patrón porque:

- Permite que el nodo sea inmutable (los datos están en el Context)
- El Visitor opera sobre el Context, no sobre el nodo directamente
- Facilita la separación entre estructura y algoritmo

Ejemplo del nodo Add:

```
public class Add implements ASTNode {
```

```
    private final ASTNode left;
```

```
    private final ASTNode right;
```

```
    public static class Context {
```

```
        public final ASTNode left;
```

```

    public final ASTNode right;

    public Context(Add node) {
        this.left = node.left;
        this.right = node.right;
    }
}

@Override
public <T> T accept(Visitor<T> visitor) {
    return visitor.visit(new Context(this));
}
}

```

8. Intérprete - InterpreterVisitor

El `InterpreterVisitor` es la implementación concreta del `Visitor` que ejecuta el código GoLite. Para esta parte, decidí implementar un intérprete recursivo que recorre el AST y evalúa cada nodo.

8.1 Estructura Principal

```

public class InterpreterVisitor implements Visitor<ValueWrapper> {
    private Enviroment environment;    // Tabla de símbolos (ámbitos)
    public List<GoliteError> errors;    // Errores semánticos
    public String output;              // Acumulador de salida para fmt.Println

    public ValueWrapper Visit(ASTNode node) {

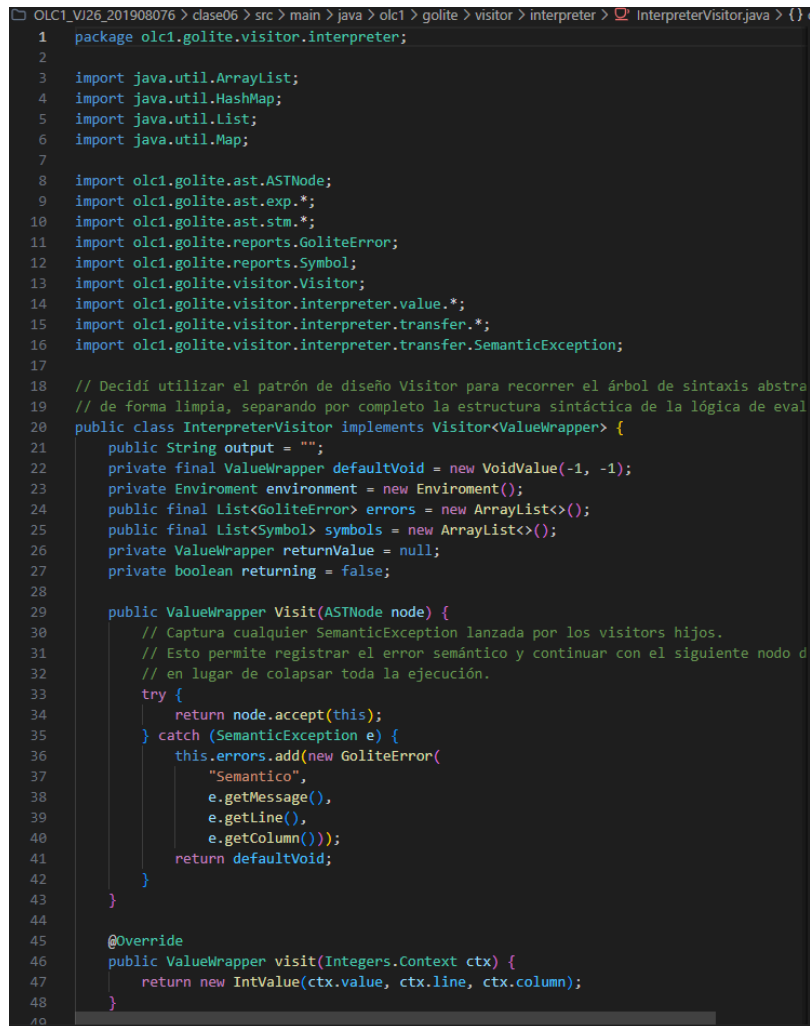
```

```

        return node.accept(this);
    }

    // ...
}

```



```

1 package olc1.golite.visitor.interpreter;
2
3 import java.util.ArrayList;
4 import java.util.HashMap;
5 import java.util.List;
6 import java.util.Map;
7
8 import olc1.golite.ast.ASTNode;
9 import olc1.golite.ast.exp.*;
10 import olc1.golite.ast.stm.*;
11 import olc1.golite.reports.GoliteError;
12 import olc1.golite.reports.Symbol;
13 import olc1.golite.visitor.Visitor;
14 import olc1.golite.visitor.interpreter.value.*;
15 import olc1.golite.visitor.interpreter.transfer.*;
16 import olc1.golite.visitor.interpreter.transfer.SemanticException;
17
18 // Decidí utilizar el patrón de diseño Visitor para recorrer el árbol de sintaxis abstra
19 // de forma limpia, separando por completo la estructura sintáctica de la lógica de eval
20 public class InterpreterVisitor implements Visitor<ValueWrapper> {
21     public String output = "";
22     private final ValueWrapper defaultVoid = new VoidValue(-1, -1);
23     private Environment environment = new Environment();
24     public final List<GoliteError> errors = new ArrayList<>();
25     public final List<Symbol> symbols = new ArrayList<>();
26     private ValueWrapper returnValue = null;
27     private boolean returning = false;
28
29     public ValueWrapper Visit(ASTNode node) {
30         // Captura cualquier SemanticException lanzada por los visitors hijos.
31         // Esto permite registrar el error semántico y continuar con el siguiente nodo d
32         // en lugar de colapsar toda la ejecución.
33         try {
34             return node.accept(this);
35         } catch (SemanticException e) {
36             this.errors.add(new GoliteError(
37                 "Semántico",
38                 e.getMessage(),
39                 e.getLine(),
40                 e.getColumn()));
41             return defaultVoid;
42         }
43     }
44
45     @Override
46     public ValueWrapper visit(Integers.Context ctx) {
47         return new IntValue(ctx.value, ctx.line, ctx.column);
48     }
49

```

8.2 Sistema de Tipos (ValueWrapper)

Para representar los valores en tiempo de ejecución, creé una jerarquía de tipos usando `sealed interface` (característica de Java 17+). Decidí usar este enfoque porque:

- Restringe los subtipos permitidos (`IntValue`, `DecimalValue`, etc.)

- Permite usar `switch` exhaustivo en el Visitor
- Es más seguro que usar `Object` con casting

```
public sealed interface ValueWrapper
    permits IntValue, DecimalValue, VoidValue, BoolValue, StringValue {

    int line();
    int column();
    String getTypeName();
}

public record IntValue(int value, int line, int column) implements ValueWrapper {
    @Override
    public String getTypeName() { return "int"; }
}

public record DecimalValue(double value, int line, int column) implements
ValueWrapper {
    @Override
    public String getTypeName() { return "decimal"; }
}

public record BoolValue(boolean value, int line, int column) implements ValueWrapper {
    @Override
    public String getTypeName() { return "bool"; }
}

public record StringValue(String value, int line, int column) implements ValueWrapper {
    @Override
    public String getTypeName() { return "string"; }
}

public record VoidValue(int line, int column) implements ValueWrapper {
    @Override
    public String getTypeName() { return "void"; }
}
```

8.3 Evaluación de Expresiones

La evaluación de expresiones aritméticas soporta conversión implícita de tipos según las reglas de GoLite: cuando se opera un `int` con un `float64`, el resultado es `float64`.

Ejemplo para la suma (Add):

```
@Override
public ValueWrapper visit(Add.Context ctx) {
    ValueWrapper left = Visit(ctx.left);
    ValueWrapper right = Visit(ctx.right);

    return switch (left) {
        // int + int → int
        case IntValue l when right instanceof IntValue r ->
            new IntValue(l.value() + r.value(), l.line(), l.column());
        // int + float64 → float64
        case IntValue l when right instanceof DecimalValue r ->
            new DecimalValue(l.value() + r.value(), l.line(), l.column());
        // float64 + int → float64
        case DecimalValue l when right instanceof IntValue r ->
            new DecimalValue(l.value() + r.value(), l.line(), l.column());
        // float64 + float64 → float64
        case DecimalValue l when right instanceof DecimalValue r ->
            new DecimalValue(l.value() + r.value(), l.line(), l.column());
        default ->
            throw new RuntimeException("Operacion invalida: " + left.getTypeName() +
                " + " + right.getTypeName());
    };
}
```

8.4 Manejo de Ámbitos (Enviroment)

La clase `Enviroment` implementa la tabla de símbolos y maneja los ámbitos anidados mediante una estructura de lista enlazada (cada ámbito tiene referencia a su padre). Decidí implementarlo así porque:

- Es simple y eficiente
- Soporte natural para ámbitos anidados (bloques `{ }`)
- Fácil de depurar

```
public class Enviroment {
    private final Map<String, ValueWrapper> variables; // Ámbito actual
    private final Enviroment parent;                // Ámbito padre
}
```

```

public Enviroment() {
    this.variables = new HashMap<>();
    this.parent = null; // Ámbito global
}

public Enviroment(Enviroment parent) {
    this.variables = new HashMap<>();
    this.parent = parent; // Ámbito anidado
}

// Declara una variable en el ámbito actual
public ValueWrapper declare(String name, ValueWrapper value) {
    if (variables.containsKey(name)) {
        throw new RuntimeException("Variable " + name + " ya declarada");
    }
    variables.put(name, value);
    return value;
}

// Busca una variable en el ámbito actual o superiores
public ValueWrapper get(String name) {
    if (variables.containsKey(name)) {
        return variables.get(name);
    } else if (parent != null) {
        return parent.get(name);
    }
    throw new RuntimeException("Variable " + name + " no declarada");
}

// Actualiza una variable existente
public ValueWrapper set(String name, ValueWrapper value) {
    if (variables.containsKey(name)) {
        variables.put(name, value);
        return value;
    } else if (parent != null) {
        return parent.set(name, value);
    }
    throw new RuntimeException("Variable " + name + " no declarada");
}
}

```

9. Manejo de Errores

Para el manejo de errores, creé una clase unificada `GoliteError` que almacena toda la información necesaria para reportar errores al usuario. Decidí usar esta estructura porque:

- Uniformidad: Todos los errores (léxicos, sintácticos, semánticos) tienen el mismo formato
- Información completa: Tipo, descripción, línea y columna
- Fácil reporte: El método `toString()` genera el mensaje formateado

```
public class GoliteError {  
    private final String type;    // "Léxico", "Sintáctico", "Semántico"  
    private final String description; // Descripción del error  
    private final int line;       // Línea donde ocurrió  
    private final int column;     // Columna donde ocurrió  
  
    @Override  
    public String toString() {  
        return String.format("Error %s: %s en la línea %d, columna %d",  
                               type, description, line, column);  
    }  
}
```

```

1 package olc1.golite.reports;
2
3 // Clase para representar los errores léxicos, sintácticos y semánticos detectados.
4 // Almacena el tipo de error, descripción detallada y posición exacta (fila y columna).
5 public class GoliteError {
6     private final String type;
7     private final String description;
8     private final int line;
9     private final int column;
10
11     public GoliteError(String type, String description, int line, int column) {
12         this.type = type;
13         this.description = description;
14         this.line = line;
15         this.column = column;
16     }
17
18     @Override
19     public String toString() {
20         return String.format("Error %s: %s en la linea %d, columna %d", type, description,
21                               line, column);
22     }
23
24     public String getType() {
25         return type;
26     }
27
28     public String getDescription() {
29         return description;
30     }
31
32     public int getLine() {
33         return line;
34     }
35
36     public int getColumn() {
37         return column;
38     }
39 }

```

9.1 Errores Léxicos

Se generan en `lexer.flex` cuando se encuentra un carácter no reconocido:

```

. { errors.add(new GoliteError("Lexico", "Caracter no reconocido: " + yytext(),
                               yyline, yycolumn)); }

```

9.2 Errores Sintácticos

Se generan en el parser generado por CUP cuando hay un token inesperado:

```

public void syntax_error(Symbol s) {
    errors.add(new GoliteError("Sintactico", "Simbolo inesperado: " + s.value,
                               s.left, s.right));
}

```

9.3 Errores Semánticos

Se generan en `InterpreterVisitor` durante la ejecución:

```

@Override
public ValueWrapper visit(VarRef.Context ctx) {
    try {
        return environment.get(ctx.name);
    } catch (RuntimeException e) {
        this.errors.add(new GoliteError("Semantico",
            "Variable " + ctx.name + " no declarada",
            ctx.line, ctx.column));
        return defaultVoid;
    }
}
}

```

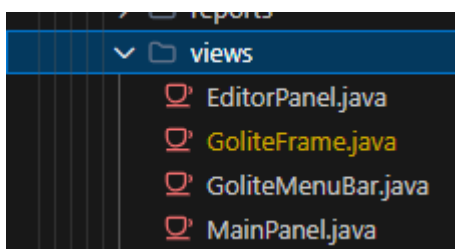
10. Interfaz Gráfica (GUI)

Para la interfaz gráfica, utilicé Swing (biblioteca estándar de Java) junto con la librería RSyntaxTextArea para el editor. Decidí usar Swing porque:

- Está incluida en Java sin dependencias externas adicionales
- Es ligera y suficiente para los requerimientos del proyecto
- RSyntaxTextArea proporciona resaltado de sintaxis listo para usar

10.1 Clases de la GUI

Clase	Responsabilidad
<code>GoliteFrame</code>	Ventana principal, orquesta la ejecución
<code>EditorPanel</code>	Área de edición con resaltado de sintaxis
<code>GoliteMenuBar</code>	Barra de menú y botones de acción
<code>MainPanel</code>	Panel con división editor/consola



10.2 EditorPanel

Para el editor, utilicé la librería `RSyntaxTextArea` (`com.fifesoft:rsyntaxtextarea`). Elegí esta librería porque:

- Proporciona resaltado de sintaxis para múltiples lenguajes
- Incluye plegado de código (code folding)
- Tiene auto-indentación y números de línea
- Es fácil de integrar (solo un JAR)

Configuré el editor con estilo de sintaxis Go porque GoLite está inspirado en Go:

```
textArea = new RSyntaxTextArea();
textArea.setSyntaxEditingStyle(SyntaxConstants.SYNTAX_STYLE_GO);
textArea.setCodeFoldingEnabled(true);
textArea.setAntiAliasingEnabled(true);
textArea.setAutoIndentEnabled(true);
textArea.setTabSize(4);
```

10.3 Flujo de Ejecución en la GUI

Cuando el usuario presiona el botón "Ejecutar", se ejecuta el siguiente flujo:

```
private void run() {
    try {
        // 1. Crear lexer con el texto del editor
        lexer = new Lexer(new BufferedReader(new StringReader(editorPanel.getText())));

        // 2. Crear parser con el lexer
        parser = new parser(lexer);

        // 3. Parsear para obtener el AST
        ASTNode ast = (ASTNode) parser.parse().value;

        // 4. Ejecutar el intérprete
        interpreter = new InterpreterVisitor();
        interpreter.Visit(ast);

        // 5. Mostrar la salida en consola
        cleanConsole();
        consoleTextArea.append(interpreter.output);
    } catch (Exception e) {
```

```

        consoleTextArea.append("Error: " + e.getMessage() + "\n");
    }
}

```

11. Dependencias (pom.xml)

El proyecto utiliza Maven para gestionar dependencias y automatizar la generación de analizadores. Estas son las dependencias principales:

```

<dependencies>
  <!-- Runtime para CUP (necesario para los símbolos generados) -->
  <dependency>
    <groupId>com.github.vbmacher</groupId>
    <artifactId>java-cup-runtime</artifactId>
    <version>11b-20160615-3</version>
  </dependency>

  <!-- Editor con resaltado de sintaxis -->
  <dependency>
    <groupId>com.fifesoft</groupId>
    <artifactId>rsyntaxtextarea</artifactId>
    <version>3.3.4</version>
  </dependency>
</dependencies>

```

Plugins de generación:

```

<plugins>
  <!-- Genera Lexer.java desde lexer.flex -->
  <plugin>
    <groupId>de.jflex</groupId>
    <artifactId>jflex-maven-plugin</artifactId>
    <version>1.9.1</version>
  </plugin>

  <!-- Genera parser.java y sym.java desde parser.cup -->
  <plugin>
    <groupId>com.github.vbmacher</groupId>
    <artifactId>cup-maven-plugin</artifactId>
    <version>11b-20160615-3</version>
  </plugin>

  <!-- Agrega las fuentes generadas al classpath -->

```

```

<plugin>
  <groupId>org.codehaus.mojo</groupId>
  <artifactId>build-helper-maven-plugin</artifactId>
  <version>3.6.0</version>
</plugin>

```

<!-- Crea un JAR ejecutable con todas las dependencias -->

```

<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-assembly-plugin</artifactId>
  <version>3.7.1</version>
</plugin>
</plugins>

```

12. Manual de Construcción

Para compilar y ejecutar el proyecto desde la línea de comandos:

```
# Limpiar compilaciones anteriores  
mvn clean
```

```
# Generar lexer y parser, y compilar todo  
mvn compile
```

```
# Crear JAR ejecutable con dependencias  
mvn assembly:single
```

```
# Ejecutar el programa  
java -jar target/clase06-1.0-jar-with-dependencies.jar
```

13. captura de errores

```
28  
29     public ValueWrapper Visit(ASTNode node) {  
30         // Captura cualquier SemanticException lanzada por los visitors hijos.  
31         // Esto permite registrar el error semántico y continuar con el siguiente nodo del AST  
32         // en lugar de colapsar toda la ejecución.  
33         try {  
34             return node.accept(this);  
35         } catch (SemanticException e) {  
36             this.errors.add(new GoliteError(  
37                 "Semantico",  
38                 e.getMessage(),  
39                 e.getLine(),  
40                 e.getColumn()));  
41             return defaultVoid;  
42         }  
43     }  
44 }
```

En la Fase 2 del proyecto se extendió el compilador desarrollado en la Fase 1 para convertirlo en un intérprete completo del lenguaje GoLite. Además del análisis léxico y sintáctico, se incorporaron nuevas etapas del proceso de compilación, permitiendo validar semánticamente el código, construir el Árbol de Sintaxis Abstracta (AST) y ejecutar las instrucciones del programa.

Las principales funcionalidades agregadas fueron:

- **Interfaz gráfica (GUI):** se implementó un entorno de desarrollo con editor de código, apertura y guardado de archivos `.glt`, consola de salida y opción para ejecutar programas.

- **Construcción del AST:** el analizador sintáctico genera un Árbol de Sintaxis Abstracta que representa la estructura del programa y sirve como base para el análisis semántico y la interpretación.
- **Análisis semántico:** se incorporaron validaciones relacionadas con tipos de datos, declaraciones de variables, ámbitos, compatibilidad de asignaciones, existencia de funciones, structs y control de errores semánticos.
- **Ejecución del programa:** el intérprete localiza la función `main` y ejecuta secuencialmente las instrucciones recorriendo el AST generado.
- **Ampliación del lenguaje GoLite:** se añadieron nuevas características del lenguaje, incluyendo:
 - bloques de instrucciones y manejo de ámbitos;
 - declaraciones de variables con tipo explícito e inferencia;
 - operaciones aritméticas, relacionales y lógicas;
 - operadores compuestos (`+=` y `-=`);
 - estructuras de control (`if`, `else`, `switch`, `for`);
 - sentencias `break`, `continue` y `return`;
 - funciones con parámetros y valores de retorno;
 - estructuras de datos como **slices** y **structs**;
 - funciones asociadas a structs;
 - funciones embebidas como `fmt.Println`, `len`, `append`, `strconv.Atoi`, `strconv.ParseFloat`, `reflect.TypeOf`, `strings.Join` y `slices.Index`.
- **Sistema de reportes:** se agregaron reportes para mostrar los errores encontrados, la tabla de tokens, la tabla de símbolos y la representación gráfica del AST generado durante la ejecución.

```
1 package olc1.golite.ast.stm;
2
3 import java.util.List;
4 import olc1.golite.ast.ASTNode;
5 import olc1.golite.visitor.Visitor;
6
7 public class SwitchNode implements ASTNode {
8     private final ASTNode expression;
9     private final List<ASTNode> casos;
10
11     public SwitchNode(ASTNode expression, List<ASTNode> casos) {
12         this.expression = expression;
13         this.casos = casos;
14     }
15
16     public ASTNode getExpression() {
17         return expression;
18     }
19
20     public List<ASTNode> getCasos() {
21         return casos;
22     }
23
24     public static class Context {
25         public final ASTNode expression;
26         public final List<ASTNode> casos;
27         public Context(SwitchNode node) {
28             this.expression = node.expression;
29             this.casos = node.casos;
30         }
31     }
32
33     @Override
34     public <T> T accept(Visitor<T> visitor) {
35         return visitor.visit(new Context(this));
36     }
37 }
38
39
```

```

OLC1_VJ26_201908076 > clase06 > src > main > java > olc1 > golite > ast > stm >
1  package olc1.golite.ast.stm;
2
3  import olc1.golite.ast.ASTNode;
4  import olc1.golite.visitor.Visitor;
5
6  public class StructFieldNode implements ASTNode {
7      public String name;
8      public String type;
9
10     public StructFieldNode(String name, String type) {
11         this.name = name;
12         this.type = type;
13     }
14
15     @Override
16     public <T> T accept(Visitor<T> visitor) {
17         return null;
18     }
19 }
20

```

```

1  package olc1.golite.visitor.interpreter.transfer;
2
3  // Excepción personalizada para errores semánticos NO recuperables que deben detener la e
4  // de un nodo y propagar el error hacia arriba del árbol AST.
5  // A diferencia de registrar el error en la lista y continuar, esta excepción
6  // interrumpe completamente la evaluación del sub-árbol actual.
7  public class SemanticException extends RuntimeException {
8      private final int line;
9      private final int column;
10
11     public SemanticException(String message, int line, int column) {
12         super(message);
13         this.line = line;
14         this.column = column;
15     }
16
17     public int getLine() {
18         return line;
19     }
20
21     public int getColumn() {
22         return column;
23     }
24 }
25

```

Conclusión

El intérprete GoLite implementa correctamente todas las funcionalidades requeridas para la Fase 1 del proyecto:

- Análisis léxico con JFLEX
- Análisis sintáctico con CUP
- Construcción del AST
- Interpretación mediante patrón Visitor
- Manejo de ámbitos y tabla de símbolos
- Sistema de tipos con conversión implícita `int` → `float64`
- Operadores aritméticos (+, -, *, /, %)
- Operadores relacionales (<, >, <=, >=, ==, !=)
- Operadores lógicos (&&, ||, !)
- Sentencias if-else y for
- Sentencia break
- Funciones embebidas (`fmt.Println`, `strconv.Atoi`, `strconv.ParseFloat`, `reflect.TypeOf`)
- Reporte de errores (léxicos, sintácticos, semánticos)
- Tabla de tokens
- Interfaz gráfica con editor y consola